SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA1711

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3 **Duration: 1 Hour**

Total Marks: 30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I **SHABARISH N 192424007** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Question 1: Doctor Shift Scheduling using Backtracking Search

Step 1: Formulating the Problem

This is a Constraint Satisfaction Problem (CSP).

- Variables: D1, D2, D3
- Domain of each variable: {Morning, Afternoon, Night}
- I am using the order: Morning = 1, Afternoon = 2, Night = 3

Constraints given in the question:

- (i) D1 ≠ Night
- (ii) D2 must come before D3, i.e. value(D2) < value(D3)
- (iii) Only one doctor per shift (all values must be different)
- (iv) D3 ≠ Morning
- (v) All 3 doctors must get exactly one shift each

Step 2: Applying Backtracking Search

I am assigning variables in the order D1, D2, D3 and trying values in the order Morning, Afternoon, Night.

Try D1 = Morning

- Check constraint (i): D1 ≠ Night → Morning is fine. Assignment accepted.

Try D2 = Morning

- Morning is already taken by D1, so constraint (iii) fails. This value is rejected.

Try D2 = Afternoon

- No conflict with D1. Assignment accepted.

Try D3 = Morning

- Constraint (iv) says D3 ≠ Morning, so this fails. Also Morning is already taken by D1. Rejected.

Try D3 = Afternoon

- Afternoon is already taken by D2, constraint (iii) fails. Rejected.

Try D3 = Night

- Constraint (ii): D2(Afternoon=2) < D3(Night=3) → True
- Constraint (iv): D3 ≠ Morning → True (it is Night)
- Constraint (iii): Morning, Afternoon, Night are all different → True

- All constraints are satisfied. This is accepted as the final answer.

Since a complete and valid assignment was found, backtracking stops here. No need to go back and try other branches.

Step 3: Final Answer

Doctor	Shift Assigned
D1	Morning
D2	Afternoon
D3	Night

Checking: D1 is not Night (ok), D2 comes before D3 (ok), all shifts are different (ok), D3 is not Morning (ok), all doctors have a shift (ok).

Python Code for Backtracking

```
shifts = ['Morning', 'Afternoon', 'Night']
order = {'Morning':1, 'Afternoon':2, 'Night':3}
doctors = ['D1', 'D2', 'D3']

def is_valid(assign):
    if assign.get('D1') == 'Night':
        return False
    if 'D2' in assign and 'D3' in assign:
        if order[assign['D2']] >= order[assign['D3']]:
            return False
    vals = list(assign.values())
    if len(vals) != len(set(vals)):
        return False
    if assign.get('D3') == 'Morning':
        return False
    return True

def backtrack(assign):
    if len(assign) == 3:
        return assign
    var = doctors[len(assign)]
    for val in shifts:
        assign[var] = val
        print('Trying', var, '=', val)
        if is_valid(assign):
            result = backtrack(assign)
            if result:
                return result
        del assign[var]
    return None

print('Final schedule:', backtrack({}))
```

Question 2: Robot Path Planning in 5x5 Grid

Step 1: Drawing the Grid

I have taken Start S at (0,0) and Goal G at (4,4). Obstacles are marked X. Rows and columns start from 0.

S	**.**	**.**	**X**	**.**
.	**X**	**.**	**X**	**.**
.	**X**	**.**	**.**	**.**
.	**X**	**X**	**X**	**.**
.	**.**	**.**	**.**	**G**

Step 2: Which Search Algorithm I Used

Normal BFS does not use a heuristic or a priority queue, it just uses a simple FIFO queue. But the question asks for Manhattan Distance heuristic and priority queue updates, so I have used A* Search here, which is basically BFS improved with g(n) + h(n) values and a priority queue. This is the correct way to include a heuristic in a BFS-type search.

Step 3: Node Expansion Table

Step	Node	g(n)	h(n)	f=g+h	Queue Update
1	(0,0) S	0	8	8	Add (1,0) and (0,1), both f=8
2	(1,0)	1	7	8	Add (2,0), f=8
3	(0,1)	1	7	8	Add (0,2), f=8
4	(2,0)	2	6	8	Add (3,0), f=8
5	(0,2)	2	6	8	Add (1,2), f=8
6	(3,0)	3	5	8	Add (4,0), f=8
7	(1,2)	3	5	8	Add (2,2), f=8
8	(4,0)	4	4	8	Add (4,1), f=8
9	(2,2)	4	4	8	Add (2,3), f=8
10	(4,1)	5	3	8	Add (4,2), f=8
11	(2,3)	5	3	8	Add (2,4), f=8
12	(4,2)	6	2	8	Add (4,3), f=8
13	(2,4)	6	2	8	Add (1,4) f=10, (3,4) f=8
14	(4,3)	7	1	8	Add (4,4), f=8
15	(4,4) G	8	0	8	Goal found, stop here

Step 4: Final Path and Cost

Path: (0,0) -> (1,0) -> (2,0) -> (3,0) -> (4,0) -> (4,1) -> (4,2) -> (4,3) -> (4,4)

Total number of moves = 8. Since each move costs 1, Total Cost = 8. This matches the heuristic value we calculated at the start (h=8), so this path is optimal.

Python Code

```
import heapq

grid = [
  ['S','.','.','#','.'],
  ['.','#','.','#','.'],
  ['.','#','.','.','.'],
  ['.','#','#','#','.'],
  ['.','.','.','.','G']
]
start, goal = (0,0), (4,4)

def h(n):
    return abs(n[0]-goal[0]) + abs(n[1]-goal[1])

def neighbors(n):
    r,c = n
    for dr,dc in [(-1,0),(1,0),(0,-1),(0,1)]:
        nr,nc = r+dr,c+dc
        if 0<=nr<5 and 0<=nc<5 and grid[nr][nc] != '#':
            yield (nr,nc)

pq = [(h(start), 0, start)]
came_from = {start: None}
cost = {start: 0}

while pq:
    f, g, node = heapq.heappop(pq)
    if node == goal:
        break
    for nxt in neighbors(node):
        new_g = g + 1
        if nxt not in cost or new_g < cost[nxt]:
            cost[nxt] = new_g
            heapq.heappush(pq, (new_g + h(nxt), new_g, nxt))
            came_from[nxt] = node

path = []
cur = goal
while cur:
    path.append(cur)
    cur = came_from[cur]
path.reverse()
print('Path:', path)
print('Cost:', len(path)-1)
```

Question 3: Autonomous Rescue Robot

(a) How the Constraints Affect Decision Making

- The robot can only move in 4 directions, so at every point it has maximum 4 choices.
- Since some cells are blocked, and the robot does not know this in advance, it can only find out by actually reaching near that cell.
- The robot can only see the cells next to it (adjacent cells), so it cannot make a full plan from the start till the end. It has to decide step by step. This is why it is called an online search problem.
- Normal cells cost 1 and risky cells cost 3 (1 + 2 penalty), so the robot cannot just count number of steps, it has to add up the real cost.
- Because the robot wants to minimize cost but also be safe, sometimes it may have to avoid a shorter path if it passes through risky zones.
- As the robot moves, it keeps updating what it knows about the map, so its plan can change in the middle of the journey.

(b) My Solution Approach: Online Uniform Cost Search

Since the map is not known fully in the beginning and the movement costs are different (normal and risky), I am using Online Uniform Cost Search, which is similar to LRTA* algorithm. The steps I followed are:

1. Robot starts at S. It does not know the full map yet.

2. At the current cell, robot senses the 4 adjacent cells and finds out if they are free, blocked or risky.

3. For each free/risky neighbor, robot calculates the cost (1 for normal, 3 for risky).

4. These neighbor cells are added to a priority queue based on total cost so far.

5. Robot picks the cell with the lowest cost from the queue and actually moves there (this is what makes it 'online', because the robot is really moving, not just simulating).

6. If the robot later finds a cell it thought was free is actually blocked, it removes that cell and goes back to try another path.

7. This process repeats until the robot reaches G. At the end we get the actual path taken and its total cost.

(c) AI Concepts Applied

Concept	Explanation
Intelligent Agent	The robot senses its surroundings (percepts) using its sensors and acts (moves) based on what it senses. This perceive-act cycle makes it an agent.
Rationality	The robot is rational because it always tries to pick the action that will help it reach the survivor while keeping the cost as low as possible and staying safe, based on what it currently knows.
Environment Type	Partially observable (only sees nearby cells), discrete (grid), deterministic (moves have fixed results), sequential (one move affects the next), and single agent.

(d) Why This Approach is the Best Choice

- Normal BFS or A* need the complete map before starting, but here the robot does not have that, so Online Search is needed.
- BFS assumes every move costs the same, but here risky zones cost more, so Uniform Cost Search (which works with different costs) is better than BFS.
- If the robot's knowledge turns out wrong (like a cell it thought was free is actually blocked), online search lets it update and re-plan, which a normal offline search cannot do.
- This method will eventually reach the goal because it keeps trying other paths when one fails, and it tries to keep the total cost as low as possible.

So, Online Uniform Cost Search (LRTA* style) fits this problem the best because it can handle partial information, different move costs, and changing knowledge at the same time.

Python Code

```
import heapq

true_grid = [
  ['S','.','R','.','.'],
  ['#','.','#','R','.'],
  ['.','.','.','#','.'],
  ['.','#','R','.','.'],
  ['.','.','.','#','G']
]
start, goal = (0,0), (4,4)
known = {}

def sense(pos):
    r,c = pos
    for dr,dc in [(-1,0),(1,0),(0,-1),(0,1)]:
        nr,nc = r+dr,c+dc
        if 0<=nr<5 and 0<=nc<5:
            known[(nr,nc)] = true_grid[nr][nc]

def step_cost(cell):
    return 3 if known.get(cell) == 'R' else 1
```

```
def online_ucs(start, goal):
    current = start
    total = 0
    path = [start]
    visited = {start}
    while current != goal:
        sense(current)
        options = []
        r,c = current
        for dr,dc in [(-1,0),(1,0),(0,-1),(0,1)]:
            nxt = (r+dr, c+dc)
            if nxt in known and known[nxt] != '#' and nxt not in visited:
                heapq.heappush(options, (step_cost(nxt), nxt))
        if not options:
            path.pop()
            current = path[-1]
            continue
        cost, nxt = heapq.heappop(options)
        total += cost
        current = nxt
        visited.add(current)
        path.append(current)
    return path, total

path, total = online_ucs(start, goal)
print('Path:', path)
print('Total cost:', total)
```

Rubric for Calculation

Criteria	Weightage	Level 4	Level 3	Level 2	Level 1
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided